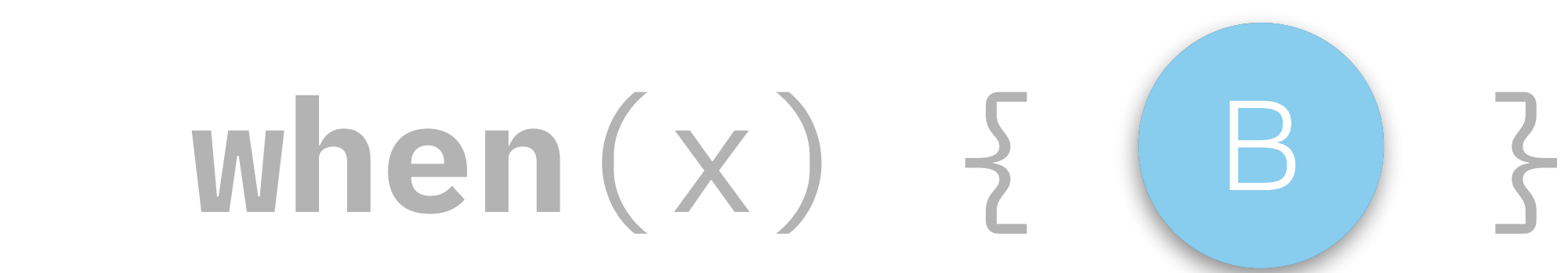
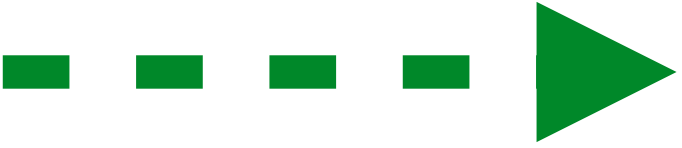

when(x) { B }

```
when(x) {
```

```
x.balance += 10;
```

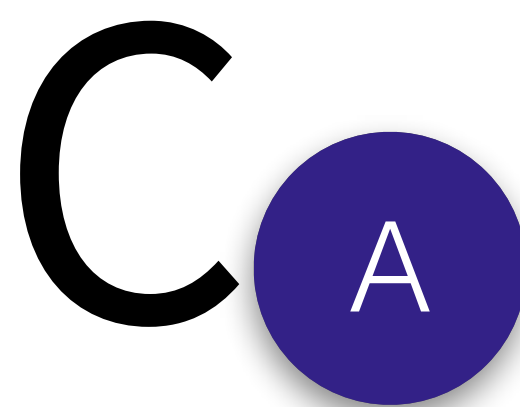


An Axiomatic Concurrency Model

49


S
A

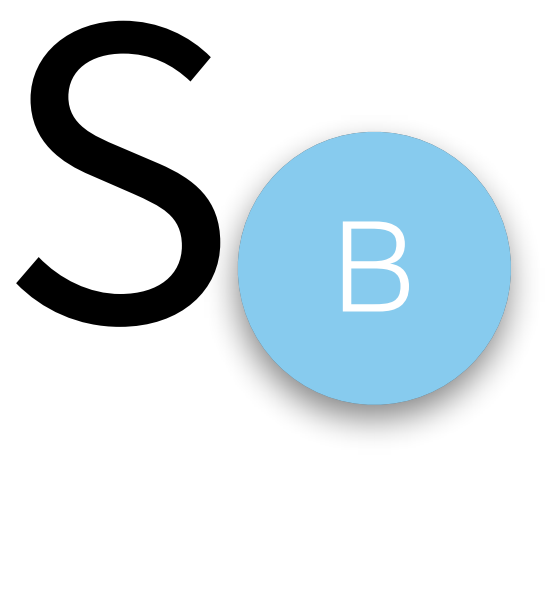

R
A


C
A

Each behaviour has three lifecycle events: Spawn, Run and Complete

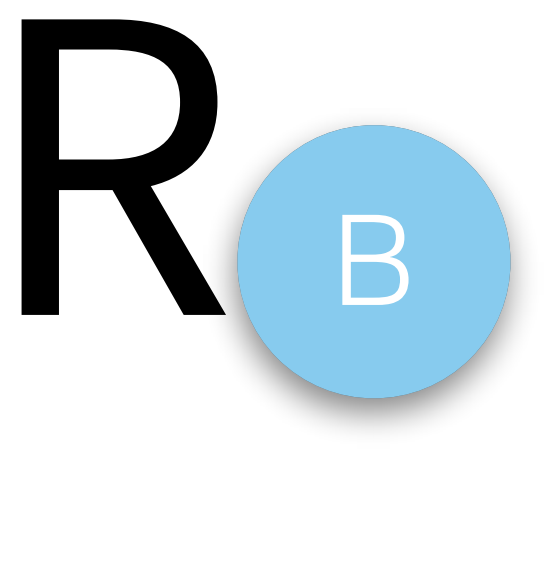
Build around candidate executions

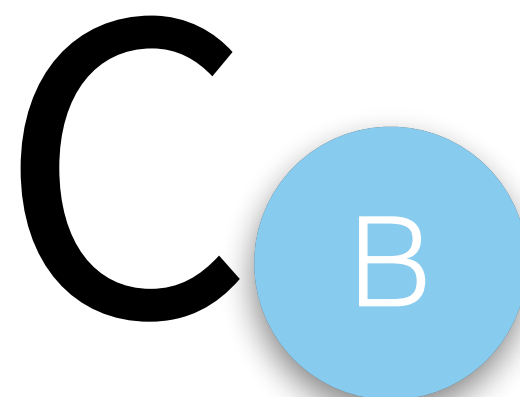
Executions are the events of the program and their order


S
B

po

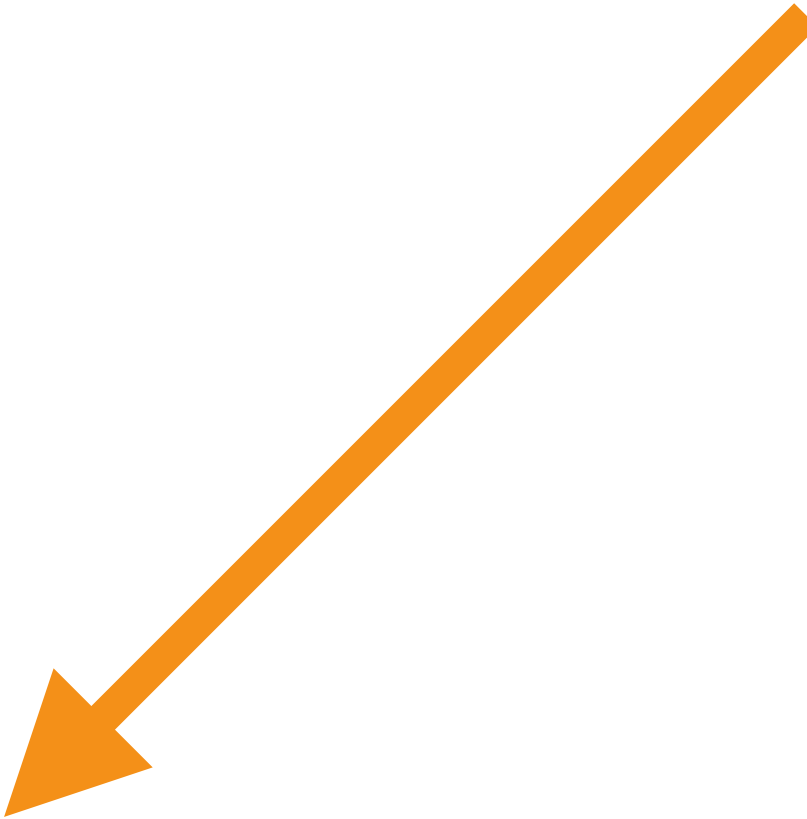

po


R
B


C
B


po

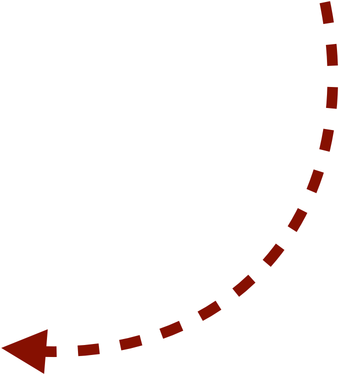
po **is the program order**



co is the cown order


CO
X

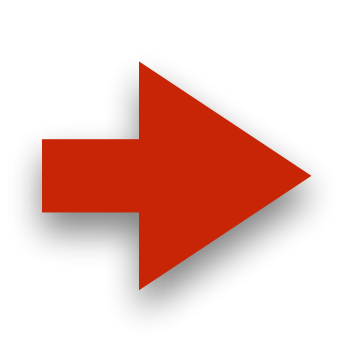
r is the derived run order

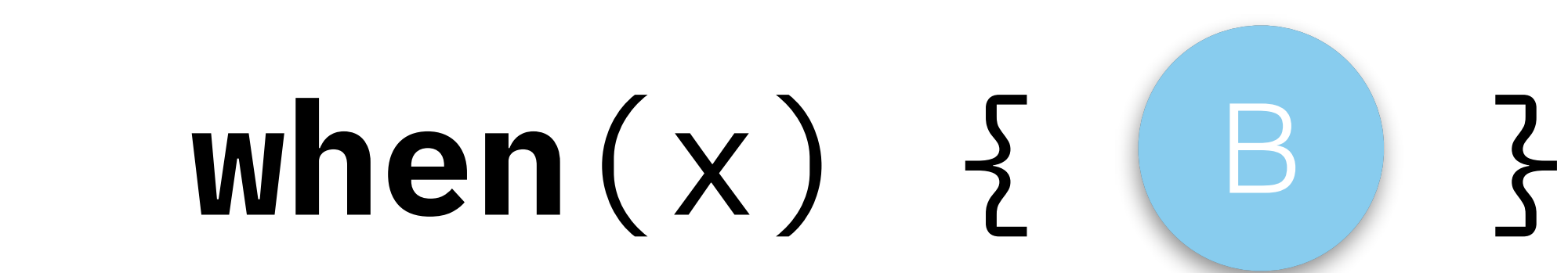



h b

hb **is the derived happens before order**

If **behaviours share a cown** and have a **path between their spawns** then **a complete happens-before a run**




when(x) { B }

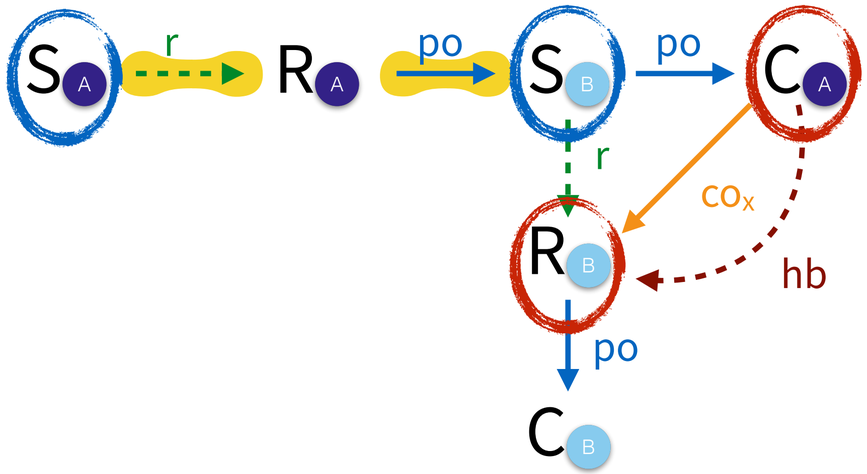
```
when(x) {
```

```
x.balance += 10;
```

Executions are valid if acyclic

An Axiomatic Concurrency Model

```
when(x) { A
  x.balance += 10;
  when(x) { B }
}
```



Build around candidate executions

Executions are the events of the program and their order

Each behaviour has three lifecycle events: Spawn, Run and Complete

po **is the program order**

co **is the cown order**

r **is the derived run order**

hb **is the derived happens before order**

If **behaviours share a cown** and have a **path between their spawns** then **a complete happens-before a run**

Executions are valid if acyclic

Validating Causality

```
def update(acc: cown[Account], amount: int) {
  when(acc) { A
    acc.balance += amount
    val id = acc.id()
    when(log) { B log.record(id, amount) }
} }
```

update(src, +100)

update(src, -100)